

OBJECT ORIENTED PROGRAMMING

● What is an object?

A person or thing that can be seen & touched & specified action or feeling is known as object

OBJECT ORIENTATION

The interaction b/w the object is known as object orientation. These objects interact with each other to perform different operations.

● What is model?

A model is an abstraction of something real or conceptual. In simple words, the collection of different objects which interact with each other & represent the reality is known as Model.

PROPERTIES OF OBJECT

- 1) Unique identity
- 2) state (attributes)
- 3) well-defined behaviour (operations)

ADVANTAGE OF OBJECT ORIENTATION

- 1) Object orientation helps to solve our real world problems.
- 2) We can easily develop an object oriented model for a problem.
- 3) Everyone can easily understand an object oriented model.
- 4) We can easily implement an object oriented model, object oriented language like C++.

INFORMATION HIDING

Information hiding means something hidden from outside world or people.

In OOP Information Hiding is: (2)
"Hiding the object details (state & behaviour) from the users"

ENCAPSULATION

Encapsulation & Information Hiding are much related concept (Information's Hiding is achieved) using encapsulation.

Encapsulation means "we have enclosed all the characteristics of an object in the object itself"

ADVANTAGE OF ENCAPSULATION

1) Simplicity & clarity

اس میں سہارا Data اور Functions اسی Object میں Save کر دیتے ہیں۔ اس سے سہارا ساڈل Simple اور Clear بن جاتا ہے۔

2) Low complexity

جب سارا Data اور Functions اسی Objects میں Save ہوتے ہیں۔ تو کوئی Complexity نہیں ہوتی۔

3) Better Understanding

جب سارا Data اور Functions اسی Object میں Save ہوتے ہیں۔ تو ہمیں اس کو سمجھنے میں بھی آسانی ہوتی ہے۔

4) Very Easy

جب سارا Data اور Functions اسی Object میں Save ہوتے ہیں تو اس ماڈل کو بنانے میں بھی آسانی ہوتی ہے۔

INTERFACES

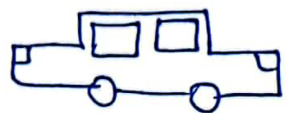
Interface is a set of functions of an object that he wants to expose to other objects.

- Interfaces are necessary for objects communication.
- Interface of an object provides us the list of available functions.

IMPLEMENTATION

It is actual implementation of the behavior of the object in any object oriented language.

Implementation is done through two things,
1) Internal Data Structure (Hidden)
2) Functionality.



Car

- 1) Car start → structure to start
- 2) Change Gear → Function to change
- 3) Apply brakes → structure to brake
- 4) lights on → function on/off.
lights off

ABSTRACTION

Capture only those details about an object that are relevant to current perspective or related to the problems."

Abstraction Example:-

"Mr Mohsin Raza is a PHD student & teaches Bachelor's & Masters students, virtually!"

ADVANTAGES OF ABSTRACTION

- 1) Help to solve problem
It helps us to understanding & solving a problem using object oriented approach as it hides extra irrelevant details of objects.
- 2) Single perspective
Focusing on single perspective of an object provides us freedom to change implementation for other aspects of for an object later.

CLASSES

All objects of same or identical characteristics or attributes we have combine all the attributes & make sketch or prototype or map this is known as "Classes"

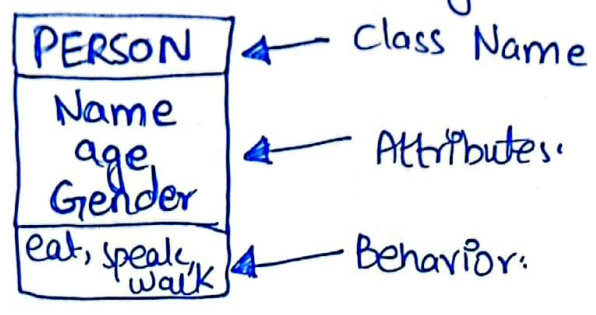
Mohsin Raza Ali Raza Ahmed Raza

Person

→ Age, Name, gender, color

REPRESENTATION OF CLASSES

We can represent a class using a rectangle as follows:



INHERITANCE

A child inherits characteristics of its parents besides inherited characteristics, a child may have its own unique characteristics.

If Class B inherits from class A then it contains all the characteristics (information, structure & behavior) of class A. The parent class is called **Base class** & the child class is called **derived class**.

ADVANTAGES OF INHERITANCE

1) Reuse

Main purpose of inheritance is reuse, we can easily add new classes by inheriting from existing classes.

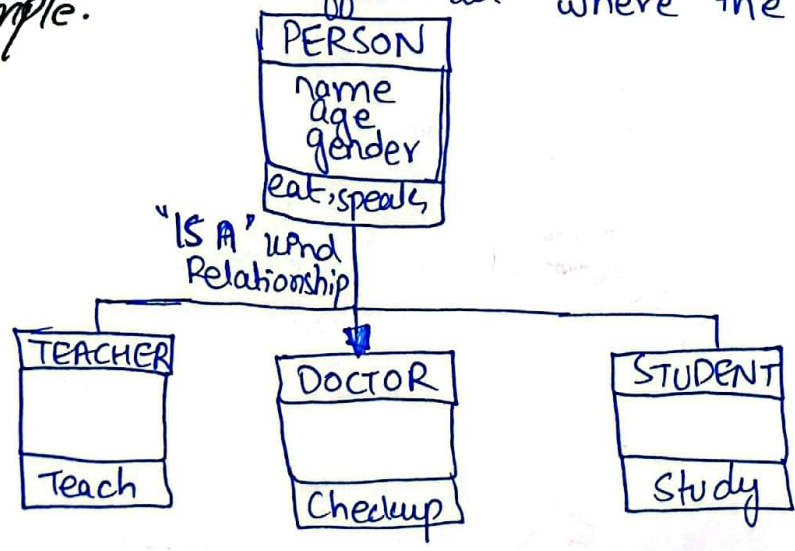
2) less Redundancy

In inheritance we cannot repeat the things, so then we have less redundancy.

3) Increased maintainability

In inheritance the maintainability is very easy. Change at one place will effect all where the used.

Example.



GENERALIZATION

(5)

There are many objects with common characteristics in an object model. The common characteristics (attributes & behavior) of all these objects are combined in a single general class. Base class encapsulates the idea of commonality of derived classes. Base class is general class representing common behavior of all derived classes. This concept is known as Generalization.

SUB-TYPING (EXTENSION)

- Sub-typing means that derived class is behaviorally compatible with base class.
- Derived class has all the characteristics of base class plus some extra characteristics.
- Behaviorally compatible ~~mean~~ means that base class can be replaced by the derived class.

GENERALIZATION VS SUB TYPING (EXTENSION)

Subtyping & generalization are related concepts, subtyping (extension) & generalization is a way to look at the same thing in two ways.

Sub typing is looking at things from Top to Bottom whereas in generalization we look at things from Bottom to Top.

SPECIALIZATION

We want to add a class to an existing hierarchy of classes having many similarities to already existing classes but some part of its behavior is different or restricted. In that case we will use the concept of specialization.

- Specialization means that derived class is behaviorally incompatible with the base class.
- Behaviorally compatibility means that base class can't always be replaced by the derived class.
- Derived class has some different or restricted characteristics than of base class.

SPECIALIZATION EXAMPLE

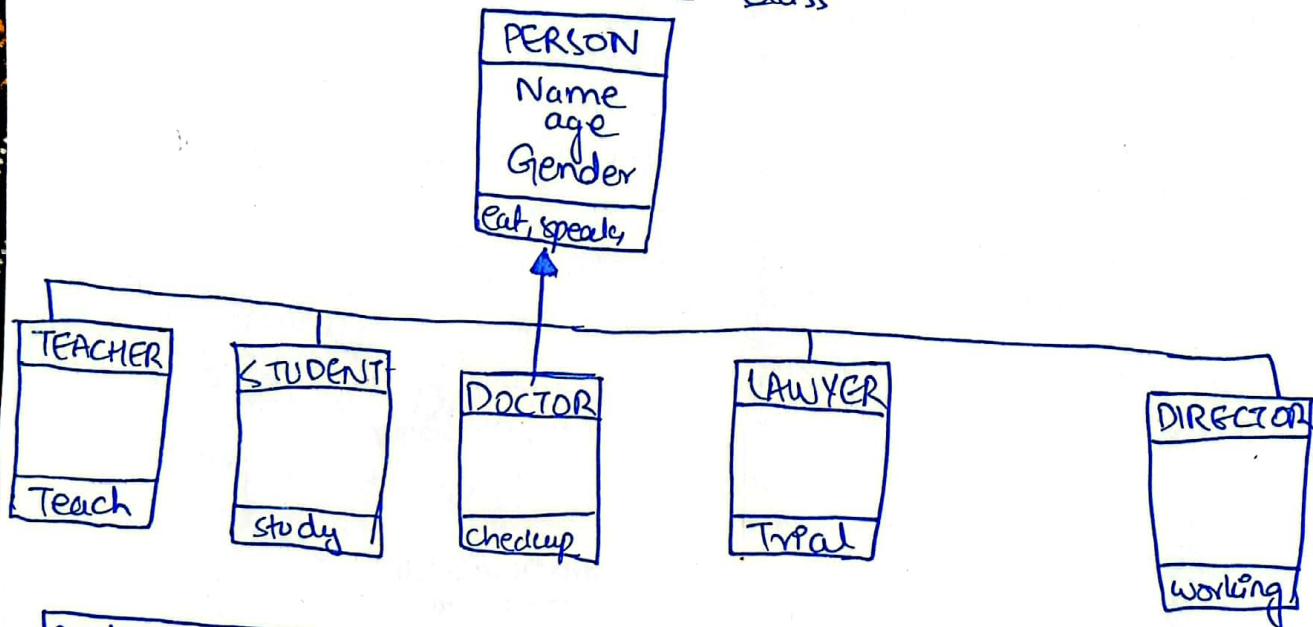
(6)

- Suppose we want to add one more class of Adult for some special requirement like for ID card generation such that it is a person but its age is greater than 18 & having all other behavior of that of person class
- We derive Adult class from person class & restrict age in that class as shown in diagram.

ABSTRACT CLASS VS CONCRETE CLASSES

Abstract classes is the class from which we derived other classes

Concrete class is the class we have derived from the base class is known as concrete class



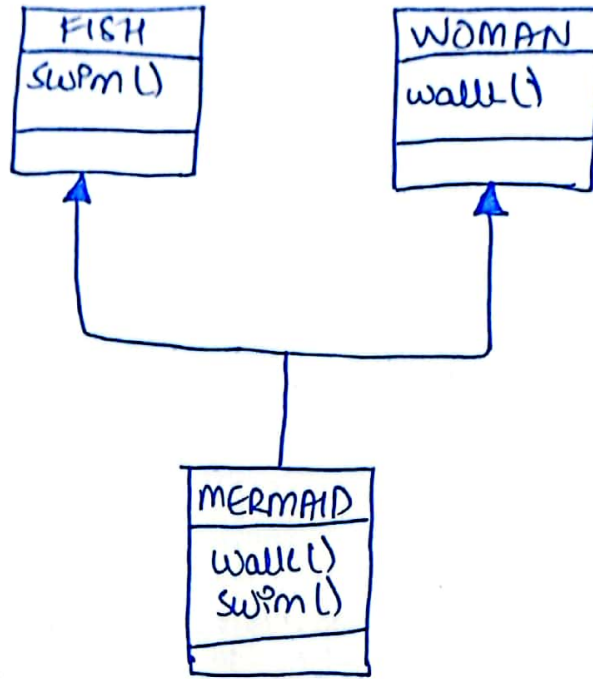
Abstract classes	PERSON				
Concrete class	TEACHER	DOCTOR	STUDENT	LAWYER	DIRECTOR

MULTIPLE INHERITANCE

Sometimes we want to reuse characteristics of more than one parent class, in that case we need to inherit a class from more than one classes. This concept is known as Multiple Inheritance

Example

(7)



CLASSES

All objects of same or identical characteristics or attributes we have combine all the attributes & make sketch or prototype or map this is known as "Class". A class includes both data members as well as member functions to manipulate that data. Normally member function are called methods.

STRUCTURE OF CLASS

ایک Class میں کچھ Data Member ایسے ہوتے ہیں جو چند User کو Accessible ہوتے ہیں ان کو ہم Public کہتے ہیں۔ اور کچھ Data Members ایسے ہوتے ہیں جو ہر User کو Access نہیں ہوتے ان کو ہم Private کہتے ہیں۔

Class class-name {

Public:

// public data members

Private:

// private data members

};

EXAMPLE:-

Write C++ program and make three classes. You must define the name of the classes as:-

- 1) Women class
- 2) Fish class
- 3) Mermaid class

And also print the methods of the classes into the main section by using multiple inheritance.

PROGRAM

```
#include <iostream>
#include <stdio>
using namespace std;
// Create women class
Class women { → Public:
    void walk() {
        cout << "WALK method of Women class";
        cout << endl;
    }
};

// Create Fish class
Class Fish {
    Public:
    void swim() {
        cout << "SWIM method of FISH class";
        cout << endl;
    }
};

// Create mermaid class
Class Mermaid:
    Public Woman, Public Fish {
};

int main () {
    Mermaid mermaid;
    mermaid.walk();
}
```

mermaid. swim()

(9)

ASSOCIATION

Interaction of different objects in OO model is known as Association. In object oriented model, objects interact with each other in order to perform some useful work, while modeling these objects (entities) is done using the association. This association can be represented with a line along an arrow head ($\rightarrow$) or without arrow head.

KINDS OF ASSOCIATION

There are two main kinds of association.

CLASS ASSOCIATION

Class association is implemented in terms of inheritance. Inheritance is considered class association.

□ In case of public inheritance it is a "IS-A" relationship

OBJECT ASSOCIATION

It is the interaction of and stand alone object of one class with other objects of another class. It can be one of the following types

□ Simple Association

□ Composition.

□ Aggregation.

SIMPLE ASSOCIATION

In Simple Association one object can interact with some other object. It is generally called as "association" instead of "Simple association"

KINDS OF SIMPLE ASSOCIATION

Simple association can be categorized in two ways

- 1) With Respect to Number of objects (Cardinality)
- 2) With respect to direction (Navigation)

KINDS OF SIMPLE ASSOCIATION W.R.T NAVIGATION (10)

1) One-Way Association

In one way Association we can navigate along a single direction only, it is denoted by an arrow towards the server object.



2) Two-Ways Association

In two ways Association we can navigate in both directions, it is denoted by a line between the associated objects.



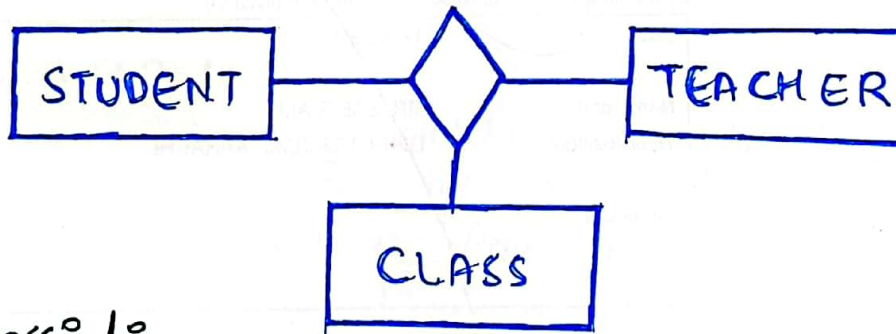
KINDS OF SIMPLE ASSOCIATION W.R.T CARDINALITY

Binary Association

It associates object of exactly two classes; it is denoted by a line, or an arrow between the associated objects.

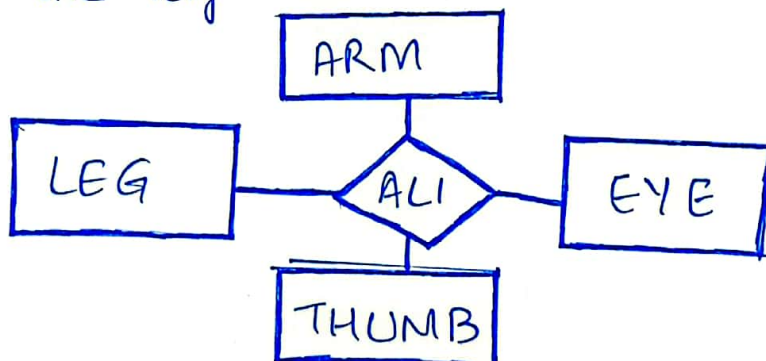
Ternary Association

It associates objects of exactly three classes; it is denoted by a diamond with lines connected to associated objects.



N-Ary Association

An Association between 3 or more classes its practical example are very rare.



COMPOSITION

(11)

An object may be composed of other smaller objects, the relationship between the "part" object and the "whole" object is known as **Composition**, composition is represented by a line with a **filled-diamond** head towards the composer object. composition is **stronger** relationship

Composition is a stronger relationship, because

- ❑ Composed object becomes a part of the composer
- ❑ Composed object can't exist independently

An example of **has-a** relationship

→ A person has Head → Zoo has a lion.

BENEFITS OF COMPOSITION

Given its broad use in the real world, it's no surprise that composition is also commonly used in carefully designed software components. when you use this concept, you can:

- ❑ Reuse Existing code
- ❑ Design clean and clear
- ❑ change the Implementation of a class
- ❑ Hide Internal code change.

AGGREGATION

An object may contain a **collection (aggregate)** of other object the relationship between the container and the contained object called **aggregation**. Aggregation is represented by a line with **unfilled-diamond** head towards the container. Aggregation is **weaker-relationship**

Aggregation is weaker relationship, because.

- ❑ Aggregate object is not a part of container.
- ❑ Aggregate object can exist independently.

BENEFIT OF AGGREGATION

(12)

In case of aggregation, which is closely related concept to composition. The Aggregation has provide following benefits like composition.

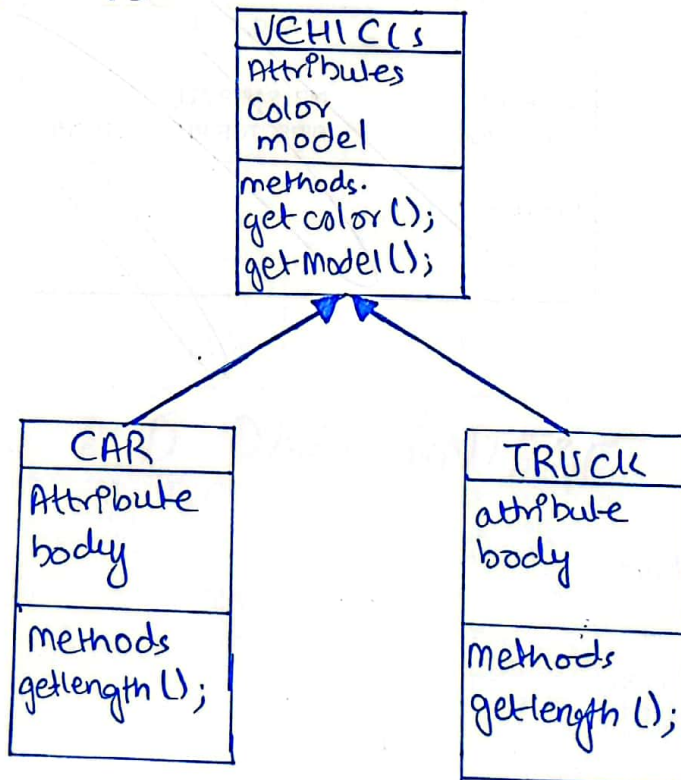
- ❑ A design has less classes
- ❑ Design clean and clear
- ❑ Aggregation means same as composition
- ❑ Aggregation is "HAS-A" Relationship.

CLASS COMPATIBILITY

A class is behaviorally compatible with another if it supports all the operations of the other class, such a class is called **subtype**. A class can be replaced by its subtype. Derived class is usually a subtype of the **base class**. It can handle all the legal messages (operation) of the base class. Therefore, base class can always be replaced by the **derived class**.

Example

Child class or derived class also provide. Includes characterstics of its base class or parent class.



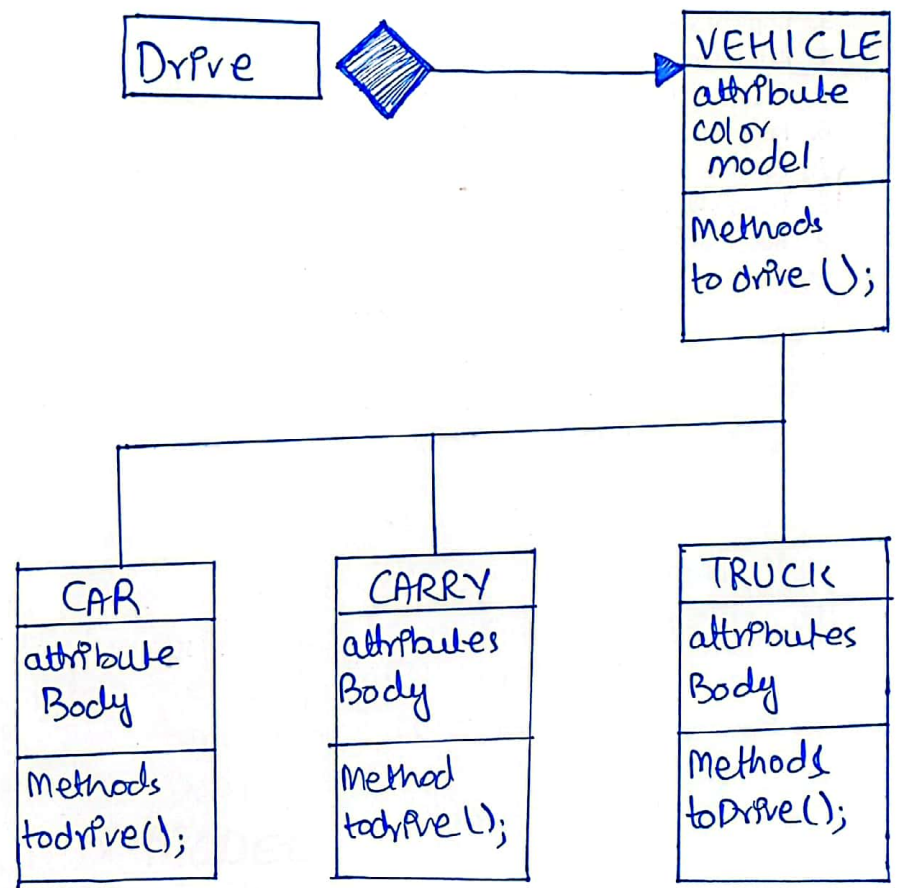
POLYMORPHISM

(13)

In OO model, polymorphism means that different object can behave in different ways. for the same message (stimulus). Consequently, sender of a message does not need to know exact class of the receiver.

Polymorphism is of two type

- **Compile Time polymorphism** - This type of polymorphism can be achieved using method overloading
- **Run Time polymorphism** - This type of polymorphism can be achieved using method ~~overloading~~ overriding & virtual function.



ADVANTAGE AND DISADVANTAGE

- Saves a lot of time
- Single variable can be used to store multiple data types
- Easy to debug the code
- It help the programmer to reuse the code, i.e classes once written, tested & implemented can be reused as required
- Polymorphism helps in reducing the coupling between different functionalities.

- ❑ The disadvantage of polymorphism is that developers find it difficult to implement polymorphism in codes. (14)
- ❑ Polymorphism reduce the readability of the program
- ❑ Run time polymorphism can lead to the performance issue as machine needs to decide which method variable to involve first at run time.

OBJECT ORIENTED MODEL

- ❑ Identifying classes
- ❑ Identify attributes
- ❑ Identify (methods) operations
- ❑ Identify Relation
- ❑ Object oriented Model
- ❑ Identify Association.
- ❑ Identify aggregation.

Example

Due to Ramadan, your nearby community wants to launch an "Online Grocery Shop" so that, people can purchase groceries from home and able to maintain social distance. This precaution will allow reducing the movement of people in Ramadan and minimize the spread of disease COVID-19. The XYZ store will offer various products. The user will be shown a list of product which can be selected or deselected from the provided list. The user will need to get registered to buy products. At the end of shopping, the customer can double-check all the products added into the cart and verify the total amount before payment. The store Administrator will manage product and assign orders to Delivery Boys.

OBJECT ORIENTED MODEL

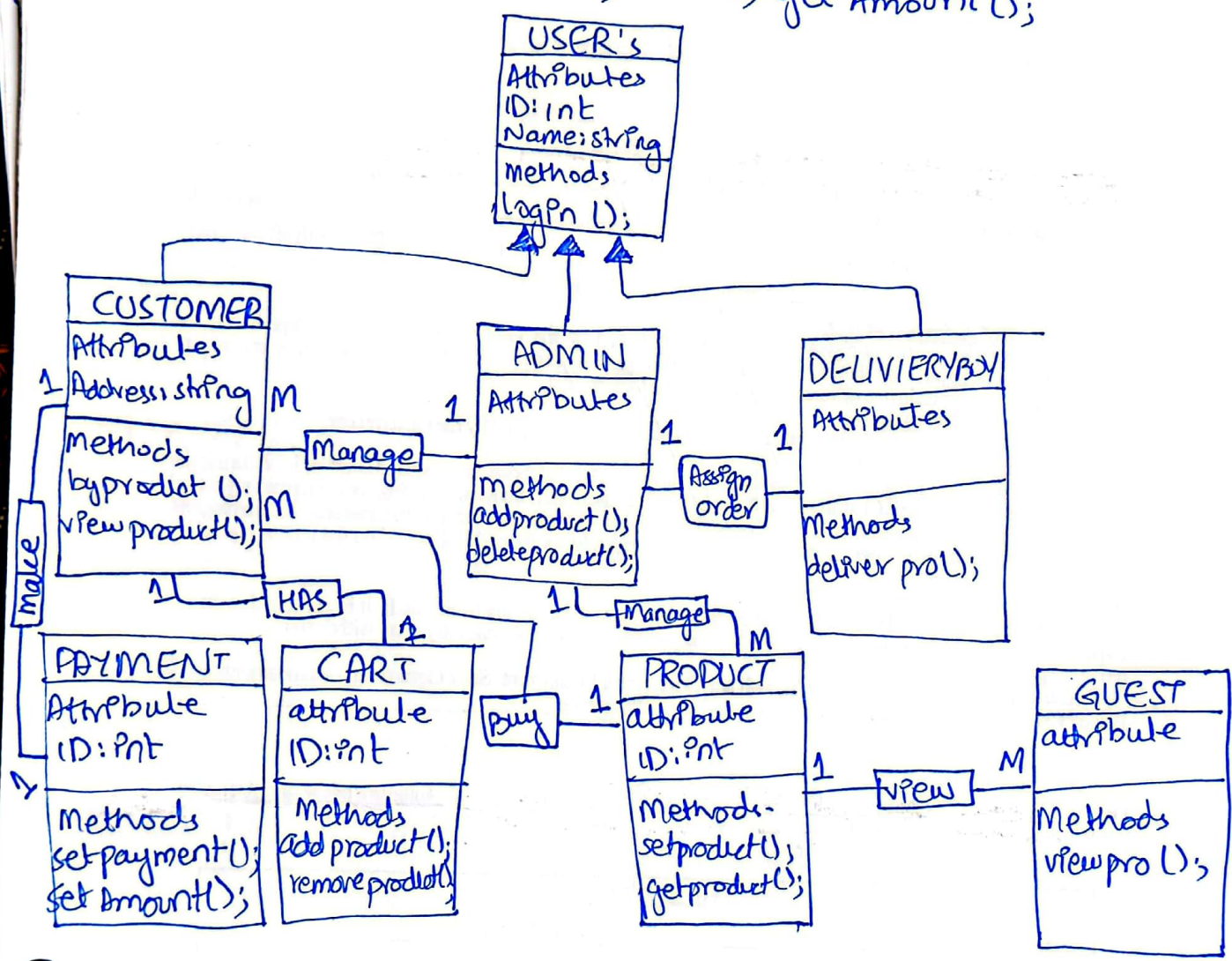
1) Identify classes

- * User * Admin * Customer * Guest * product * cart
- * payment * Delivery boy.

2) Identify Attributes

- * User (admin, customer, guest, Delivery boy)
 - 1) ID: int 2) Name: string 3) Pass: string
- * Product (1) ID: int 2) Name: string
- * Cart 1) ID: int 2) Items: string

- * Payment 1) ID: int 2) Amount: int
- 3) Identify (Methods) Operation
- * Users (Admin, Customer, Guest, Delivery boy)
 - (1) Login (); (2) order (); (3) view (); (4) Deliver ();
- * Product
 - 1) getproduct (); 2) setproduct ();
 - 3) deleteproduct (); 2) get Amount ();
- * Cart
 - 1) addproduct (); 2) setAmount ();
- * Payment



CLASSES

A Class is a "user defined" data type in which we have store different type of data . It has data member functions Normally member functions are called methods.

OBJECTS

(16)

□ When we create variable of a class, a special name is used for them. i.e. objects "Instances of a class are called objects".

STRUCTURE OF A CLASS

A class may consist of the following

□ Class Definition □ Data Members □ Member Function.

□ Access Specifier

These are used to enforce access restrictions to members of a class, there are three access specifiers.

□ 'Public' is used to tell that member can be accessed whenever you have access to the object.

□ 'private' is used to tell that member can only be accessed from a member function

□ 'protected' to be discussed when we cover inheritance.

STRUCTURE

```
Class class_name {  
    // definition of class  
};
```

```
Class student {
```

```
    private:
```

```
    char* name;
```

```
    int rollNo;
```

Data Member
Cannot Access from
outside

```
    public:
```

```
        void setName(char*);
```

```
        void setRollNo(int);
```

Member function
Cannot Access from
outside

```
    protected:
```

```
};
```

SCOPE RESOLUTION OPERATOR

In C++, scope resolution operator is :: it is used for following purposes

- ❑ To access a global variable when there is a local variable with same name
- ❑ To define a function outside a class.

Class student {

Private:

char* name;

int rollNo;

Public:

void setName(char*);

void setRollNo(int);

};

Scop Resolution operator

student::display();

class name

function

ACCESSING MEMBERS

members of an object can be accessed using

- ❑ dot operator (.) to access via the variable name.
- ❑ arrow operator (->) to access via a pointer to an object.

Class student {

Private:

char* Name;

int rollNo;

Public:

void setName(char*);

void setRollNo(int);

};

student astudent; // declaring object

astudent.rollNo = 5;

student* astudent = newstudent(); // declaring pointer.

astudent->rollNo = 5;

CONSTRUCTOR

Constructor is a special function, called whenever we access an object of a class. If we do not define a constructor function in a class, the C++ provides a default constructor. It is executed at the same time of instantiating an object. The constructor are used to initialize the data members of an object.

Types of constructor

- Compiler Generated constructor
- Simple constructor
- Parameterized constructor
- One parameterized constructor.
- Two parameterized constructor
- Three parameterized constructor.

Constructor properties.

- Constructor does not have return type.
- Constructor are commonly public members.
- Constructor is a special function having same name as the class name

COMPILER GENERATED CONSTRUCTOR

If a constructor is not defined by the user the compiler generates it automatically. This constructor has no parameter. It does nothing although the compiler will create a default constructor for us, the behavior of the compiler synthesized constructor is rarely what we want thus the default constructor provided by the compiler does no initialization for us.

Class Date {

```
    int day;
    int month;
    int year;
};
```

// compiler Generated constructor of the class

```
Date();
```

SIMPLE CONSTRUCTOR

We have earlier discussed that we can write a constructor that takes no argument. The user defined constructor, that takes no argument is called a simple constructor. We know that when a compiler generated default constructor is called, it does no initialization. It does not know whether to put a value in data member like day month in our previous class data.

```
Class Date {
```

```
    int day;
```

```
    int month;
```

```
    int year;
```

```
};
```

```
// Simple constructor of the class
```

```
    Date ();
```

PARAMETERIZED CONSTRUCTOR

We may define a constructor, which takes arguments as well. This constructor will be automatically called when the required number of arguments are passed to it. Through this we can easily assign the passed values to our class data member for that particular object.

```
Class Date {
```

```
    int day;
```

```
    int month;
```

```
    int year;
```

```
};
```

```
// parameterized constructor of class.
```

```
    Date (int, int, int);
```

DESTRUCTOR

The name of the destructor is the same as that of a class with the preceding tilde sign (~). The ~ and name of the class is written as a single word without any space between them. So the name of the destructor of class Date will be ~Date. The destructor can not be overloaded. There will be only one destructor of a class. The destructors take no argument. The destructor doesn't return a value.

CONSTRUCTOR OVERLOADING

Constructor overloading means having more than one constructor with the same name. or overloaded constructor have the same name (name of the class) but the different number of arguments. Constructors are methods invoked with an object is created. You have to use the same name for all the constructors which is the class name. This is done by declaration the constructor with a different number of argument.

```
Class Date {
    int day;
    int month;
    int year;
};
```

// Constructor overloading

```
1. Date(); 1. Date();
2. Date(int day, int month);
3. Date(int, int);
```

DEFAULT COPY CONSTRUCTOR

A copy constructor is a member function that initializes an object using another object of the same class. In simple terms, a constructor which creates an object by initializing it with an object of the same class, which has been created previously is known as a copy constructor.

In C++, a copy constructor may be called in following cases

- ☐ when an object of the class is returned by value.
- ☐ when the compiler generates a temporary object
- ☐ when an object is constructed based on another object of the same class
- ☐ when an object of the class is passed (to a function) by values as an argument.

```
Class Sample {
    int ID;
};
```

// Copy constructor

```
int main() {
    Sample obj1;
    obj1 = obj1 (10);
```

```
obj1.display();
Sample obj2(obj1);
obj2.display();
}
```

USER DEFINED COPY CONSTRUCTOR

A user-defined copy constructor is generally needed when an object owns pointers, or non-pointers shareable reference such as to a file, in which case a destructor and an assignment operator should also be written

```
Class Sample {
    int ID;
};
```

// user defined copy constructor.

```
Sample(); // default constructor
```

```
Sample(Sample &t) { // copy constructor
    id = t.id;
}
```

SHALLOW COPY

□ When we initialize one object with another then the compiler copies state of one object to the other using copy constructor by assigning data member values of previous object to newly created object

□ This kind of copying is called shallow copy

□ Shallow copy work fine if our class doesn't include dynamic memory allocation but in case of dynamic memory allocation it leads to hanging pointer problem. This problem is solved through deep copy

DEEP COPY

We write deep copy code in copy constructor so that when we create new object from an existing object using copy constructor we also allocate new dynamic memory for data members involving dynamic memory.

ACCESSOR FUNCTION:

(22)

Accessor functions are also used to access private data of the object, we provide accessor function to get and set private data members of the class

```
Class Box {  
    private:  
        int value;  
    public:  
        Box(int);  
    // Accessors  
    int getValue() const;  
};
```

- An accessor doesn't need arguments.
- An accessor has the same type as the retrieved variable
- The name of the accessor begins with the Get prefix
- A naming convention is necessary.

MUTATOR FUNCTION

- While an accessor function makes a data member accessible it does not make it editable. Modification of a protected data member requires a mutator function.
- Because they provide direct access to protected data, mutator and accessor functions must be written and used carefully.

```
Class Box {  
    private:  
        int value;  
    public:  
        Box(int);  
    // Accessors  
    int getValue() const;  
    // Mutator (non-accessor)  
    void setValue(int);  
};
```

THIS POINTER

Every object in C++ has access to its own address through an important pointer called **this pointer**. The **this pointer** is an implicit parameter to all member functions. Therefore inside a member function, **this** may be used to refer to the involving object.

To understand **this pointer**, it is important to know how objects look at functions and data members of a class.

- Each object gets its own copy of the data member
- All access the same function definition as present in the code segment

Class Text {

Private:

int *x;

Public:

void setX (int x);

// The 'this' pointer is used to retrieve the object's x

this → x = x;

void print () {

cout << "x = ";

cout << x << endl;

}

};